TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA

08

// Unidad II · Desarrollo Backend con Java

PROBAR APIS CON POSTMAN

LOS VERBOS HTTP EN ACCIÓN

EN ESTA LECTURA

01 ¿Qué es Postman?
02 Su importancia en Spring Boot
03 Instalación y primer vistazo
04 Anatomía de una petición
05 Los verbos HTTP, paso a paso
06 Leer respuestas y resolver errores
07 Colecciones y variables
08 Pruebas automáticas

ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología
Francisco “El Chele” Moreno

Fundamentos de Desarrollo
Backend con Java
Noviembre 2026

1

CAPÍTULO 01

¿QUÉ ES POSTMAN?

> **Postman** es una aplicación gratuita para **enviar peticiones HTTP** a una API y **ver sus respuestas** sin tener que programar un cliente. Es el banco de pruebas del desarrollador backend.

En la Lectura 07 aprendimos que toda comunicación con una API es un par **petición → respuesta**. Quien hace la petición es un **cliente**: una app móvil, una página web o el navegador. El problema es que, mientras construimos el backend, **todavía no existe ese cliente**. ¿Cómo comprobamos entonces que nuestros endpoints funcionan? Con Postman: él se hace pasar por el cliente.

EL CONTROL REMOTO DE TU API

Imagina que construiste un televisor (tu backend) pero aún no existe el control remoto oficial (la app). Postman es un **control remoto universal**: tiene un botón para cada verbo HTTP, te deja escribir exactamente qué canal quieres (la URL) y te muestra en pantalla lo que respondió el televisor.

¿Qué puedes hacer con Postman?

Función	Para qué te sirve
Enviar cualquier verbo	GET POST PUT PATCH DELETE con un clic, cada uno con su URL, parámetros, encabezados y cuerpo.
Ver la respuesta completa	Código de estado, tiempo de respuesta, tamaño, encabezados y el cuerpo JSON ya formateado.
Guardar peticiones	Agruparlas en **colecciones** para repetirlas cuando quieras, sin volver a escribirlas.
Usar variables	Escribir `{{baseUrl}}` una vez y cambiar de servidor (local o producción) en un solo lugar.
Automatizar pruebas	Escribir pequeñas comprobaciones que revisan solas si la respuesta es la esperada.
Compartir y documentar	Exportar la colección como archivo `.json` y subirla a GitHub junto con el proyecto.

Postman y sus alternativas

Postman es la herramienta más conocida, pero no la única. Todas hacen lo mismo en esencia: **enviar peticiones HTTP**. Lo que aprendas aquí te sirve en cualquiera de ellas.

Herramienta	Tipo	Cuándo usarla
Postman	App de escritorio y web	La más completa: colecciones, variables, pruebas y trabajo en equipo.

Herramienta	Tipo	Cuándo usarla
Thunder Client	Extensión de VS Code	Probar rápido sin salir del editor.
REST Client	Extensión de VS Code	Escribir las peticiones en un archivo `.http` que se guarda junto al código.
Insomnia / Bruno	Apps de escritorio	Alternativas ligeras a Postman con funciones parecidas.
curl	Terminal	Viene instalado en Linux, macOS y Windows; ideal para scripts y servidores sin interfaz gráfica.

2

CAPÍTULO 02

SU IMPORTANCIA EN SPRING BOOT

> Una API hecha con Spring Boot **no tiene pantallas:** solo recibe peticiones y devuelve JSON. Postman es la forma de **ver y tocar** lo que acabamos de programar.

Cuando escribimos un `@RestController`, el resultado no es una página bonita, sino un conjunto de **endpoints** esperando peticiones. Para saber si funcionan necesitamos algo que les hable. El navegador ayuda un poco, pero se queda corto muy rápido:

Necesito…	Navegador	Postman	¿Por qué?
Probar un `GET`	✓ Sí	✓ Sí	Escribir la URL en la barra de direcciones hace un GET.
Probar `POST`, `PUT`, `DELETE`	✗ No	✓ Sí	El navegador no tiene forma de elegir otro verbo desde la barra de direcciones.
Enviar un cuerpo JSON	✗ No	✓ Sí	Necesario para todo `@RequestBody`.
Ver el código de estado	Escondido	✓ A la vista	En Postman el `201` o el `404` aparece en grande junto a la respuesta.
Agregar encabezados	✗ No	✓ Sí	`Content-Type`, tokens de seguridad, etc.
Guardar y repetir pruebas	✗ No	✓ Sí	Colecciones y variables, sin reescribir nada.

El ciclo de trabajo del desarrollador backend

Postman se convierte en parte del ciclo diario. Cada vez que agregas o cambias un endpoint, lo pruebas antes de seguir:

1 · Programar Escribes el endpoint en el controlador → **2 · Ejecutar** `./mvnw spring-boot:run` → **3 · Probar** Envías la petición en Postman → **4 · Corregir** Si la respuesta no es la esperada, vuelves al paso 1

Cinco razones por las que Postman es clave

- **Probar sin frontend.** El equipo de backend no tiene que esperar a que exista la app o la página web para saber si su código funciona.
- **Verificar el contrato.** En la Lectura 07 escribimos el contrato de la API (verbo, ruta, cuerpo y respuestas posibles). Con Postman comprobamos que cada fila de ese contrato se cumple de verdad.
- **Depurar errores.** Si la app móvil "no funciona", reproduces la misma petición en Postman y descubres si el problema está en el backend o en el cliente.
- **Probar los casos de error.** Es muy fácil enviar a propósito datos inválidos, ids que no existen o JSON mal escrito para ver que la API responda `400` o `404` y no se caiga con un `500`.
- **Documentar y compartir.** Una colección exportada le dice a cualquier compañero exactamente cómo usar tu API.

POSTMAN NO REEMPLAZA LAS PRUEBAS EN CÓDIGO

En el proyecto `firstapi` del repositorio hay pruebas con **MockMvc** (`EndpointsTests.java`) que se ejecutan con `./mvnw test`. Las dos cosas se complementan: **Postman** sirve para explorar y probar a mano mientras desarrollas; las **pruebas en código** se ejecutan solas cada vez que compilas y evitan que algo que ya funcionaba se rompa sin que te des cuenta.

3

CAPÍTULO 03

INSTALACIÓN Y PRIMER VISTAZO

> Postman se descarga gratis desde **postman.com/downloads**. Usa siempre la **aplicación de escritorio** para probar tu API local.

Instalación

Sistema	Cómo instalarlo
Windows / macOS	Descarga el instalador desde **postman.com/downloads**, ábrelo y sigue los pasos.
Linux (Ubuntu)	Desde la tienda de aplicaciones (Ubuntu Software) buscando **Postman**, o en la terminal: `sudo snap install postman`

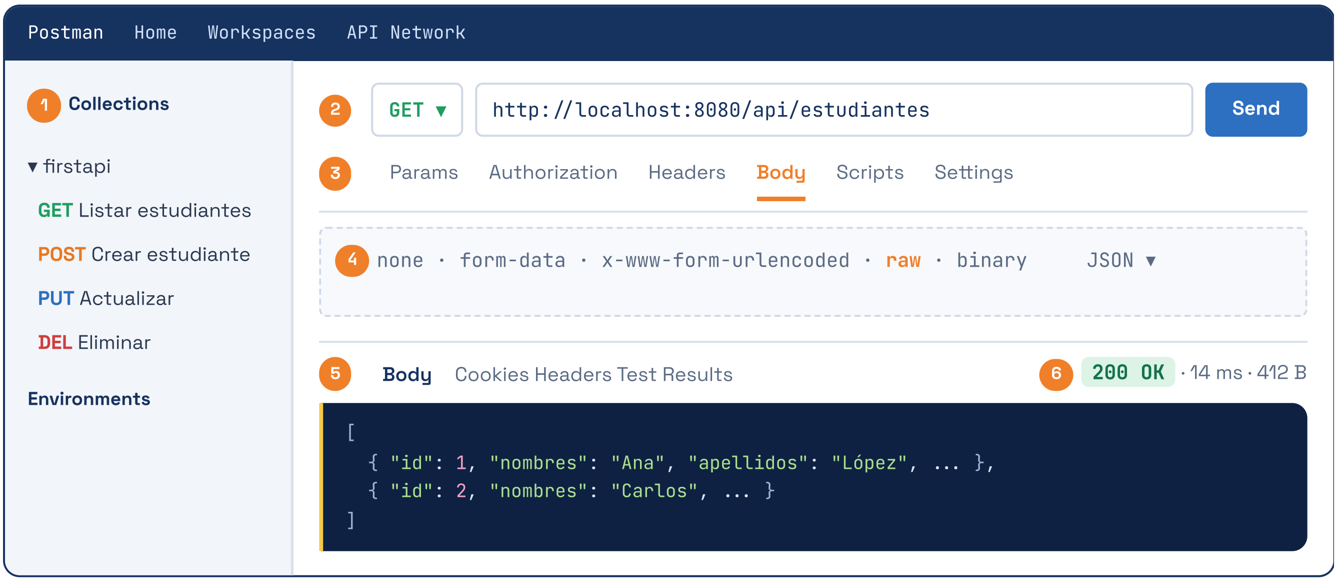
Al abrirlo por primera vez te pedirá iniciar sesión. Puedes crear una **cuenta gratuita**, que guarda tus colecciones en la nube y las sincroniza entre computadoras.

¿VERSIÓN WEB O DE ESCRITORIO?

Postman también funciona en el navegador, pero la versión web **no puede llegar a** `localhost` por sí sola, porque la petición sale desde los servidores de Postman y no desde tu computadora. Para probar la API que corre en tu máquina usa la **app de escritorio**, o instala el **Postman Agent** si prefieres la web.

Las partes de la ventana

La interfaz tiene muchas opciones, pero para empezar solo necesitas conocer seis zonas:


Postman Home Workspaces API Network
① Collections
▾ firstapi
GET Listar estudiantes
POST Crear estudiante
PUT Actualizar
DEL Eliminar
Environments
②
GET ▾
Send
③
Params Authorization Headers Body Scripts Settings
④
none · form-data · x-www-form-urlencoded · raw · binary JSON ▾
⑤
Body Cookies Headers Test Results
⑥
200 OK · 14 ms · 412 B

http://localhost:8080/api/estudiantes

```
[
  { "id": 1, "nombres": "Ana", "apellidos": "López", ... },
  { "id": 2, "nombres": "Carlos", ... }
]
```

#	Zona	Para qué sirve
①	**Barra lateral**	Tus **colecciones** (carpetas de peticiones guardadas) y tus **entornos** de variables.
②	**Barra de petición**	El **verbo** (menú desplegable), la **URL** y el botón **Send** que envía la petición.
③	**Pestañas de la petición**	**Params** (parámetros de consulta), **Headers** (encabezados), **Body** (cuerpo), **Authorization** y **Scripts** (pruebas automáticas).
④	**Editor del cuerpo**	Donde escribes el JSON. Para una API REST elige siempre **raw** + **JSON**.
⑤	**Panel de respuesta**	El cuerpo devuelto (en modo **Pretty** se ve con sangría y colores) y los encabezados de la respuesta.
⑥	**Estado de la respuesta**	**Código de estado**, tiempo que tardó y tamaño. Es lo primero que debes mirar.

4

CAPÍTULO 04

ANATOMÍA DE UNA PETICIÓN

> Cada parte de una petición HTTP tiene su lugar en Postman, y cada una llega a una **anotación distinta** en el controlador de Spring Boot.

En la Lectura 07 vimos que un endpoint puede recibir datos en tres lugares: la **ruta**, los **parámetros de consulta** y el **cuerpo**. Esta tabla une las tres piezas: lo que escribes en Postman, cómo viaja en la petición y cómo lo lee Spring Boot.

En Postman	Ejemplo	En Spring Boot	Notas
Menú del verbo	POST	`@PostMapping`	Debe coincidir con la anotación del método o recibirás `405`.
URL (ruta)	`/api/estudiantes/2`	`@PathVariable`	El `2` llega al parámetro `id`.
Pestaña Params	`nombre` = `an`	`@RequestParam`	Postman lo agrega solo a la URL: `?nombre=an`
Pestaña Headers	`Content-Type: application/json`	(lo usa Spring)	Le dice a Spring que el cuerpo es JSON.
Pestaña Body	`{ "nombres": "Mario", … }`	`@RequestBody`	Spring convierte el JSON en un objeto `Estudiante`.

Del controlador a Postman

Mira cómo se lee un método del controlador para saber **qué escribir en Postman**. Este es el endpoint de crear del proyecto `firstapi`:

firstapi/…/controller/EstudianteController.java

```java
@RestController
@RequestMapping("/api/estudiantes")          // ① la ruta base va en la URL
public class EstudianteController {

    @PostMapping                              // ② el verbo: POST
    public ResponseEntity<Estudiante> crear(
            @Valid @RequestBody Estudiante nuevo) { // ③ espera un JSON en Body
        ...
        return ResponseEntity.status(HttpStatus.CREATED).body(nuevo); // ④ 201
    }
}
```

#	Lo que dice el código	Lo que haces en Postman
1	`@RequestMapping("/api/estudiantes")`	URL: `http://localhost:8080/api/estudiantes`
2	`@PostMapping`	En el menú del verbo eliges **POST**
3	`@RequestBody Estudiante`	**Body** → **raw** → **JSON** y escribes un objeto con los atributos de `Estudiante`
4	`HttpStatus.CREATED`	Esperas ver `201 Created` en el estado de la respuesta

LOS NOMBRES DEL JSON IMPORTAN

Las claves del JSON deben llamarse **exactamente igual** que los atributos de la clase Java (respetando mayúsculas y minúsculas). Si envías `"nombre"` pero el atributo se llama `nombres`, Spring no lo reconoce, lo ignora y el campo llega vacío (`null`). En `firstapi`, eso hace que `@Valid` responda `400` porque los nombres son obligatorios.

5

CAPÍTULO 05

LOS VERBOS HTTP, PASO A PASO

> Vamos a recorrer todo el contrato del **CRUD de estudiantes** del proyecto `firstapi`, un verbo a la vez. Las respuestas que verás son **reales**, copiadas de la API funcionando.

Antes de empezar

1. Abre una terminal en la carpeta `firstapi` del repositorio y ejecuta `./mvnw spring-boot:run` (en Windows, `mvnw.cmd spring-boot:run`). Espera el mensaje `Tomcat started on port 8080`.
2. En Postman pulsa **+** (o **New** → **HTTP**) para abrir una petición nueva.
3. La API arranca con tres estudiantes de ejemplo: **Ana** (id 1), **Carlos** (id 2) y **Lucía** (id 3). Se guardan en memoria, así que **al reiniciar la app vuelven a ser solo esos tres.**

GET Leer todos los estudiantes

Elige **GET**, escribe la URL y pulsa **Send.** No hace falta cuerpo: un GET solo pide datos.

PETICIÓN · POSTMAN

```
GET  http://localhost:8080/api/estudiantes
```

Params, Headers y Body: vacíos.

RESPUESTA · 200 OK

```
[
  { "id": 1, "nombres": "Ana",
    "apellidos": "López",
    "correo": "ana@correo.com", "edad": 16 },
  { "id": 2, "nombres": "Carlos", ... },
  { "id": 3, "nombres": "Lucía", ... }
]
```

GET Filtrar con un parámetro de consulta

Ve a la pestaña **Params** y escribe la clave `nombre` con el valor `an`. Fíjate cómo Postman agrega `?nombre=an` a la URL automáticamente. En el controlador, ese valor llega al `@RequestParam String nombre`.

```
PETICIÓN · POSTMAN
GET  …/api/estudiantes?nombre=an
Params → Key: nombre · Value: an
```

```
RESPUESTA · 200 OK
[
  { "id": 1, "nombres": "Ana",
    "apellidos": "López",
    "correo": "ana@correo.com", "edad": 16 }
]
```

GET Leer un estudiante por su id

El id va **dentro de la ruta** y llega al `@PathVariable Long id`. Prueba un id que existe y otro que no:

```
PETICIÓN · id existente
GET  …/api/estudiantes/2
RESPUESTA · 200 OK
{ "id": 2, "nombres": "Carlos",
  "apellidos": "Martínez",
  "correo": "carlos@correo.com",
  "edad": 17 }
```

```
PETICIÓN · id inexistente
GET  …/api/estudiantes/99
RESPUESTA · 404 Not Found
(cuerpo vacío)
El controlador devolvió ResponseEntity.notFound().
El código de estado dice la verdad: ese estudiante
no existe.
```

POST Crear un estudiante

1. Cambia el verbo a **POST** y deja la URL base: `http://localhost:8080/api/estudiantes`
2. Pestaña **Body** → marca **raw** → en el menú de la derecha elige **JSON**. Postman agrega solo el encabezado `Content-Type: application/json`.
3. Escribe el JSON **sin** el `id`: lo asigna el servidor. Pulsa **Send**.

```
PETICIÓN · BODY (raw · JSON)
POST  …/api/estudiantes
{
  "nombres": "Mario",
  "apellidos": "Ruiz",
  "correo": "mario@correo.com",
  "edad": 16
}
```

```
RESPUESTA · 201 Created
{
  "id": 4,
  "nombres": "Mario",
  "apellidos": "Ruiz",
  "correo": "mario@correo.com",
  "edad": 16
}
El servidor le asignó el id 4. Si vuelves a pulsar
Send, se crea otro estudiante con id 5: POST no es
idempotente.
```

POST Crear con datos inválidos

Ahora rompe las reglas a propósito: nombres vacíos, un correo sin `@` y una edad menor que 14. Las anotaciones `@NotBlank`, `@Email` y `@Min` del modelo, junto con `@Valid`, rechazan la petición **antes** de que llegue a la lista.

PETICIÓN · BODY (raw · JSON)

POST …/api/estudiantes

```
{
  "nombres": "",
  "apellidos": "Ruiz",
  "correo": "no-es-correo",
  "edad": 10
}
```

RESPUESTA · `400 Bad Request` (resumida)

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for
     object='estudiante'. Error count: 3",
  "errors": [ ... ],
  "path": "/api/estudiantes"
}
```

Dentro de "errors" busca "defaultMessage": ahí están tus mensajes, como "La edad mínima es 14 años".

PUT Actualizar un estudiante

PUT **reemplaza todos los datos** del recurso, así que debes enviar el objeto completo, no solo el campo que cambia. El id va en la ruta.

PETICIÓN · BODY (raw · JSON)

PUT …/api/estudiantes/1

```
{
  "nombres": "Ana María",
  "apellidos": "López",
  "correo": "ana@correo.com",
  "edad": 17
}
```

RESPUESTA · `200 OK`

```
{
  "id": 1,
  "nombres": "Ana María",
  "apellidos": "López",
  "correo": "ana@correo.com",
  "edad": 17
}
```

Haz un GET a /api/estudiantes/1 para confirmar que el cambio quedó guardado.

DELETE Eliminar un estudiante

PETICIÓN · POSTMAN

DELETE …/api/estudiantes/3

Sin cuerpo. Solo el id en la ruta.

RESPUESTA · `204 No Content`

(cuerpo vacío)

Éxito, pero no hay nada que devolver. Si repites el mismo DELETE obtendrás 404: el estudiante ya no existe.

PATCH ¿Y si uso un verbo que no existe en el controlador?

El controlador de `firstapi` no tiene `@PatchMapping`. Si envías **PATCH** a `/api/estudiantes/1`, la ruta existe pero no acepta ese verbo, y Spring responde `405 Method Not Allowed` con el mensaje `"Method 'PATCH' is not supported."`. Es una buena forma de comprobar que el verbo forma parte del endpoint.

El contrato, comprobado

Verbo	Ruta	Body en Postman	Debes ver
GET	`/api/estudiantes`	—	`200` lista (filtro opcional `?nombre=`)

Verbo	Ruta	Body en Postman	Debes ver
`GET`	`/api/estudiantes/{id}`	—	`200` estudiante · `404` no existe
`POST`	`/api/estudiantes`	raw · JSON	`201` creado · `400` datos inválidos
`PUT`	`/api/estudiantes/{id}`	raw · JSON	`200` actualizado · `404` · `400`
`DELETE`	`/api/estudiantes/{id}`	—	`204` eliminado · `404` no existe

6

CAPÍTULO 06

LEER RESPUESTAS Y RESOLVER ERRORES

> Cuando algo falla, la respuesta de Postman es una pista. Primero mira el **código de estado**; después, el **cuerpo**; y si hace falta, la **consola de Spring Boot** en la terminal.

Cómo leer el panel de respuesta

- **Código de estado:** el primer dígito te dice de quién es el problema. `2xx` todo bien · `4xx` la petición que enviaste está mal · `5xx` falló el código del servidor.
- **Body:** usa la vista **Pretty** para ver el JSON ordenado. En los errores, lee el campo `"message"`.
- **Headers:** revisa, por ejemplo, el `Content-Type` de la respuesta.
- **Tiempo:** si una petición sencilla tarda varios segundos, algo en el backend está trabajando de más.

Errores comunes y cómo resolverlos

Lo que ves	Causa probable	Solución
Error: connect ECONNREFUSED 127.0.0.1:8080	La aplicación Spring Boot **no está corriendo**, o usa otro puerto.	Ejecuta `./mvnw spring-boot:run` y revisa `server.port` en `application.properties`.
`404 Not Found`	La **ruta está mal escrita** (`/api/estudiante` sin la ”s”), o el id no existe.	Compara la URL con `@RequestMapping` y `@GetMapping`. Si el cuerpo está vacío, fue el controlador quien dijo ”no existe”.
`405 Method Not Allowed`	La ruta existe, pero **no con ese verbo**.	Revisa el menú del verbo en Postman y la anotación del método.
`415 Unsupported Media Type`	El cuerpo no se envió como JSON (falta `Content-Type: application/json`).	En Body elige **raw** y, a la derecha, **JSON** (no *Text*).
`400` ”JSON parse error”	El JSON está **mal escrito**: una coma de más, comillas faltantes.	Postman subraya en rojo el error. Recuerda que las claves van entre comillas dobles.

Lo que ves	Causa probable	Solución
`400` "Validation failed"	Los datos **no cumplen las validaciones** (`@NotBlank`, `@Email`, `@Min`...).	Lee los `defaultMessage` dentro de `"errors"` y corrige los datos.
`500 Internal Server Error`	Una **excepción no controlada** en tu código (por ejemplo, un `NullPointerException`).	Mira la terminal donde corre Spring Boot: el error completo aparece ahí, con la línea exacta.

¿POR QUÉ LA RESPUESTA DE ERROR ES TAN LARGA?

Mientras desarrollas, **DevTools** hace que Spring incluya en los errores un campo `"trace"` con toda la pila de llamadas de Java. Es muy útil para depurar, pero no te asustes por su tamaño: busca primero `"status"`, `"error"` y `"message"`. En producción ese campo no se muestra.

7

CAPÍTULO 07

COLECCIONES Y VARIABLES

> Escribir la misma petición una y otra vez cansa. Las **colecciones** guardan tus peticiones y las **variables** evitan repetir datos como la URL del servidor.

Crear una colección

1. En la barra lateral pulsa **Collections** → **+** y nómbrala **firstapi**.
2. Con una petición abierta, pulsa **Save** (o `Ctrl + S`), dale un nombre claro como *"Crear estudiante"* y elige la colección.
3. Repite con cada endpoint del contrato. Puedes crear carpetas dentro de la colección, por ejemplo *Hello* y *Estudiantes*.

Variables: escribe la URL una sola vez

Si mañana la API se publica en un servidor real, tendrías que cambiar la URL en todas las peticiones. Con una variable solo cambias un valor. En Postman las variables se escriben entre **dobles llaves**:

SIN VARIABLES

```
http://localhost:8080/api/estudiantes
http://localhost:8080/api/estudiantes/2
http://localhost:8080/api/estudiantes/3
// la misma base repetida en todas
```

CON LA VARIABLE baseUrl

```
{{baseUrl}}/api/estudiantes
{{baseUrl}}/api/estudiantes/2
{{baseUrl}}/api/estudiantes/3
// baseUrl = http://localhost:8080
```

1. Haz clic en la colección **firstapi** y abre la pestaña **Variables**.
2. Crea la variable `baseUrl` con el valor `http://localhost:8080` y guarda.

3. En tus peticiones reemplaza el inicio de la URL por `{{baseUrl}}`. Si pasas el mouse por encima, Postman te muestra su valor.

Tipo de variable	Cuándo usarla
De colección	Datos que comparten todas las peticiones de una colección, como `baseUrl`. Es la más sencilla para empezar.
De entorno (Environment)	Cuando tienes varios servidores: un entorno *Local* con `http://localhost:8080` y otro *Producción* con el dominio real. Cambias de uno a otro con un menú.
Globales	Valores disponibles en todas tus colecciones. Úsalas con cuidado.

Compartir la colección con el equipo

Haz clic en los **tres puntos** de la colección → **Export** y guarda el archivo `firstapi.postman_collection.json` dentro de la carpeta del proyecto. Al subirlo a GitHub (Lectura 05), cualquier compañero puede importarlo con **Import** y tendrá todas las peticiones listas para usar.

NUNCA SUBAS SECRETOS

Si una petición usa contraseñas, tokens o claves de acceso, guárdalos en variables y **no** los incluyas en la colección que subes a GitHub. Un token publicado en internet debe considerarse robado y cambiarse de inmediato.

8

CAPÍTULO 08

PRUEBAS AUTOMÁTICAS

> Postman puede revisar las respuestas por ti. Escribes pequeñas **pruebas** en JavaScript y, cada vez que pulsas Send, te dice cuáles pasaron y cuáles fallaron.

Las pruebas se escriben en la pestaña **Scripts** → **Post-response** de cada petición (en versiones anteriores de Postman la pestaña se llamaba **Tests**). Se ejecutan **después** de recibir la respuesta. No necesitas saber JavaScript a fondo: la estructura siempre es la misma, y Postman ofrece fragmentos ya hechos (*snippets*) en el panel derecho.

Scripts → Post-response · petición "Crear estudiante" (POST)

```
// pm.test("descripción", función) define una prueba con nombre
pm.test("Responde 201 Created", function () {
    pm.response.to.have.status(201);
});

pm.test("El servidor asignó un id", function () {
    const estudiante = pm.response.json();     // convierte el cuerpo en objeto
    pm.expect(estudiante.id).to.be.a("number");
});

pm.test("Devuelve los mismos nombres que enviamos", function () {
    pm.expect(pm.response.json().nombres).to.eql("Mario");
});
```

Al pulsar **Send**, abre la pestaña **Test Results** del panel de respuesta. Cada prueba aparece en verde (**PASS**) o en rojo (**FAIL**) con su descripción.

◆ Ejecutar toda la colección de una vez

Con la opción **Run collection** (clic derecho sobre la colección), Postman envía **todas** las peticiones en orden y muestra un resumen de las pruebas. En segundos compruebas el contrato completo de la API, ideal antes de entregar un trabajo o de subir cambios a GitHub.

EL MISMO ESPÍRITU QUE MOCKMVC

Compara este script con `EndpointsTests.java` del proyecto `firstapi`: `pm.response.to.have.status(201)` hace lo mismo que `andExpect(status().isCreated())`. Cambian la herramienta y el lenguaje, pero la idea es igual: **enviar una petición y comprobar que la respuesta es la esperada.**

✓

PRÁCTICA

MANOS A LA OBRA

1. Crea la colección **firstapi** con la variable `baseUrl` y guarda en ella una petición por cada fila del contrato del capítulo 5.
2. Agrega una carpeta **Hello** con las peticiones a `/hello`, `/hello/{nombre}`, `/saludo?nombre=…`, `/persona` y `/personas`.
3. Provoca a propósito cada error de la tabla del capítulo 6 y anota qué código de estado recibiste.
4. Escribe al menos una prueba automática en cada petición del CRUD y ejecuta **Run collection**.
5. Exporta la colección y súbela a tu repositorio de GitHub junto con el proyecto.

Resumen

Concepto	Recuerda
Postman	Un cliente HTTP: envía peticiones a tu API y te muestra la respuesta completa.
Importancia	Una API de Spring Boot no tiene pantallas; Postman permite probar todos los verbos sin esperar al frontend.
Petición	Verbo + URL (`@PathVariable`) + Params (`@RequestParam`) + Headers + Body (`@RequestBody`).
Cuerpo JSON	Body → **raw** → **JSON**. Las claves deben llamarse igual que los atributos de la clase.
Verbos	`GET` lee · `POST` crea · `PUT` reemplaza · `PATCH` modifica · `DELETE` elimina.
Errores	Mira primero el código de estado. 404 ruta o id · 405 verbo · 415 falta JSON · 400 datos · 500 revisa la terminal.
Colecciones	Guardan tus peticiones; con `{{baseUrl}}` la URL del servidor se escribe una sola vez.
Pruebas	`pm.test(...)` en Scripts → Post-response comprueba solo la respuesta; Run collection prueba todo.

Referencias

Postman. *Postman Learning Center: Send API requests and get response data.* learning.postman.com/docs

Postman. *Write scripts to test API response data.* learning.postman.com/docs/tests-and-scripts

Postman. *Store and reuse values using variables.* learning.postman.com/docs/sending-requests/variables

MDN Web Docs. *HTTP request methods* y *HTTP response status codes.* developer.mozilla.org/es/docs/Web/HTTP

Spring. *Building a RESTful Web Service.* spring.io/guides/gs/rest-service

MARVIN RAFAEL MEZA PINEDA
Facilitador · Fundamentos de Desarrollo Backend con Java
Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026